polispec, exploratory peer review

Evidence-based feedback and policy scenario simulation | 14 October 2026

Prepared for the maintainer of SteveBenner/polispec. Independent and constructive, no affiliation or audit certification implied.

Executive assessment

polispec addresses a meaningful engineering problem: separating declared policy, agent actions, promotion authority, and evidence of enforcement. The repository documents important trust distinctions, including the difference between generated harness artifacts and verified native enforcement. That is a strong architectural direction.

The principal concern is qualification completeness, not a demonstrated universal design defect. A 13-case simplified policy-rule simulation matches the modeled example rules, but provides no evidence that polispec, any native harness, or an operator workflow will enforce those outcomes. Separately, the maintainer's published smoke records explicitly document a post-write hook failure in which prohibited content landed before an unsuccessful callback.

Scope, sources and evidence labels

Reviewed repository files: README.md, examples/policy.shop.yml, docs/harness-controls.md, docs/operator.md and smoke.yml, fetched from the public repository on 14 October 2026. The smoke log is first-party evidence and was not independently reproduced. Full source-code inspection, pinned revision checkout, native harness execution, privileged host setup, and adversarial runtime tests were not performed.

Labels: DOCUMENTED = repository claims or configuration; REPORTED = maintainer-recorded observation; SIMULATED = independent simplified rule model; UNVERIFIED = requires real execution. No claim below should be interpreted as external penetration-test certification.

Findings

F-01 | Post-write failure path | HIGH priority, observed in maintainer record

[REPORTED] smoke.yml reports a post-write private-key literal landing on disk, while its PostToolUse path produced a hook_error and no denial. The direct code check still flagged the file. Reproduce with actual installed hooks, establish whether the unsafe artifact can persist, and decide whether a pre-write or compensating rollback boundary is required.

F-02 | Enforcement default is advisory | MEDIUM deployment risk

[DOCUMENTED] README.md states polispec.enforce defaults to advise, where decisions do not block. Verify activation gates and make the active enforcement mode unmistakable in onboarding, CI, and operator reporting. An advisory deployment must not be presented as protective.

F-03 | Harness-native guarantees remain incomplete | HIGH qualification priority

[DOCUMENTED] docs/harness-controls.md, dated 13 Oct 2026, distinguishes artifact generation and parser acceptance from native enforcement. It explicitly identifies missing or limited end-to-end coverage across several harnesses, including Codex callback failure behavior. Maintain a capability-by-version matrix and executable denial journeys.

F-04 | Example policy permits warning on production data writes | CONTEXTUAL

[DOCUMENTED] examples/policy.shop.yml marks prod data.write as warn, not deny. This may be intentional for operator-controlled workflows; verify that a separate boundary prevents an ordinary agent from altering live user data. Do not treat example rules as a deployment-ready secure baseline without context.

F-05 | Rule-model consistency, 13 scenarios | LIMITED positive

[SIMULATED] A simplified first-match model of the published shop rules returned the expected verdict for 13 synthetic requests. This checks interpretation of the example configuration only. It does not implement polispec parsers, authenticated authority, trust refs, receipt verification, hooks, actual tools, or fail-closed behavior.

Scenario matrix, simplified model

Scenario	Expected example-policy result	Status
S01 Agent edits production code	DENY	SIMULATED
S02 Agent reads production secret	DENY	SIMULATED
S03 Agent pushes production branch	DENY	SIMULATED
S04 Agent writes live user data	WARN	SIMULATED
S05 Agent commits directly in test	DENY	SIMULATED
S06 Agent deploys to test	ALLOW	SIMULATED
S07 Agent promotes to test with gates satisfied	ALLOW	SIMULATED
S08 Agent promotes to test with failed gates	DENY (GATE)	SIMULATED
S09 Agent copies PII to development	DENY	SIMULATED
S10 Agent edits policy in test	WARN	SIMULATED
S11 Agent rewrites main history	WARN	SIMULATED
S12 Agent changes dev source file	ALLOW	SIMULATED
S13 Agent asks to promote production	DENY	SIMULATED

Important: S08 models a failed promotion gate abstractly, not the real promotion gate machinery. S09 assumes the data-class crossing was correctly recognized. S13 models an agent-level direct action, not authenticated operator promotion.

Recommended qualification roadmap

P0 | Reproduce failure and prove prevention

Pin exact commit and harness versions, enable enforce mode, reproduce the post-write secret case and a pre-write control, inspect the resulting filesystem and exit/event traces. Expected pass means prohibited content never persists, or a documented corrective action prevents downstream use.

P0 | Test authority-boundary failure modes

Exercise malformed events, hook timeout, callback process death, missing/modified trusted policy, installed receipt corruption, renamed protected paths, symlink traversal, direct shell equivalents, and stale approval replay. All tests must assert observed side effects, not just returned verdicts.

P1 | Cross-harness execution matrix

Repeat concrete allow/deny/ask flows on supported versions of Claude Code and Codex first, then Hermes and Antigravity where actual runtimes are available. Mark absent runtimes NOT TESTED rather than PASS; distinguish parser acceptance and enforcement.

P1 | Race, recovery, and consistency

Stress concurrent tool calls, interrupted installation, atomic mutation and rollback, trusted-ref drift, and event/receipt integrity. Ensure the safety boundary survives interruptions.

P2 | Benchmark usability and overhead

Measure denial true-positive/false-positive rates, policy evaluation latency, agent workflow disruption, recovery guidance, and maintenance overhead. Repeat with representative legitimate tasks.

Acceptance standard and maintainer questions

For each case, preserve: scenario ID, policy and trust-ref hashes, implementation commit, runtime and harness versions, environment, exact tool event, expected/observed decision, resulting side effect, raw log checksum, cleanup outcome, and reviewer conclusion. Separate PASS, FAIL, NOT TESTED, and INCONCLUSIVE. A reported block with an already-written forbidden file is a FAIL for preventive enforcement.

Questions to resolve together: Is post-write checking intended to be preventive or detective? Which harness/version combinations are genuinely enforced? What is the fail-closed policy when hooks fail? Which production operations intentionally remain warn-only, and which independent controls constrain them? What evidence is necessary before a lock or enforcement claim is made?

Overall conclusion

Promising architecture and unusually candid documentation of verification gaps. The project would benefit most from a smaller, harder, reproducible enforcement matrix before claiming broad native protections. This assessment is intended as actionable peer feedback, not a critique of development velocity or the maintainer personally.

Primary references

https://github.com/SteveBenner/polispec/blob/main/README.md

https://github.com/SteveBenner/polispec/blob/main/examples/policy.shop.yml

https://github.com/SteveBenner/polispec/blob/main/docs/harness-controls.md

https://github.com/SteveBenner/polispec/blob/main/docs/operator.md

https://github.com/SteveBenner/polispec/blob/main/smoke.yml